

Core Java & Java 8+

Complete Interview Questions Guide

50+ Interview Questions with Detailed Answers

Perfect for Interview Preparation

Shaikh Shakir

Java Developer

PART 1

Core Java

OOPs, Logic & Memory Management

This section covers 25+ essential Core Java interview questions

↓ 25+ Questions Inside ↓

OOPs Fundamentals

Question 1

Explain the 4 pillars of Object-Oriented Programming.

Answer:

Pillar	Description	Real-world Example
Abstraction	Hiding complex implementation details and showing only essential features	ATM machine - you insert card, enter PIN, get money; you don't know internal processing
Encapsulation	Binding data and methods together, hiding data using private access	Class with private variables and public getter/setter methods
Inheritance	Child class acquiring properties of parent class	Dog class extends Animal class
Polymorphism	One interface, multiple implementations	Same method name behaving differently in different classes

Code Example:

```
// Abstraction
abstract class Vehicle {
    abstract void start();
}

// Encapsulation
class Person {
    private String name;
```

```
    public String getName() { return name; }  
    public void setName(String name) { this.name = name; }  
}  
  
// Inheritance  
class Car extends Vehicle {  
    void start() { System.out.println("Car starts"); }  
}  
  
// Polymorphism  
Vehicle v = new Car();  
v.start(); // Car starts
```

💡 **Interview Tip:** Always give real-world examples along with code. This shows deep understanding.

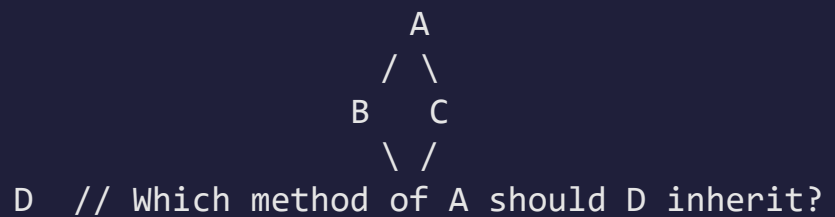
Diamond Problem

Question 2

What is the diamond problem in inheritance, and how does Java solve it?

Answer:

The diamond problem occurs when a class inherits from two classes that have a common ancestor, causing ambiguity about which parent's method to call.



Java's Solution:

- Java doesn't allow multiple inheritance of classes
- Multiple inheritance is allowed only through interfaces
- If two interfaces have same default method, implementing class must override it

Code Example:

```
interface A {  
    default void show() { System.out.println("A"); }  
}  
  
interface B {  
    default void show() { System.out.println("B"); }  
}
```


Inheritance Types

Question 3

What types of inheritance does Java support? Why is multiple inheritance not supported for classes?

Answer:

Supported Inheritance Types:

- **Single** - One parent, one child
- **Multilevel** - Grandparent → Parent → Child
- **Hierarchical** - One parent, multiple children

Not Supported:

- Multiple inheritance (two parents)
- Hybrid inheritance

Why no multiple inheritance?

- **Ambiguity** - Diamond problem
- **Complexity** - Method resolution becomes complex
- **Java designers** chose simplicity over complexity

```
// Single Inheritance
class Parent { }
class Child extends Parent { }

// Multilevel Inheritance
class GrandParent { }
class Parent extends GrandParent { }
```


final, finally, finalize

Question 4

Difference between final, finally, and finalize?

Keyword	Purpose	When Used
final	Restricts modification	Variables: constant value Methods: cannot override Classes: cannot extend
finally	Cleanup block	Always executes after try-catch, regardless of exception
finalize()	Garbage collection	Called by GC before object is destroyed (deprecated in Java 9+)

Code Example:

```
// final variable
final int MAX = 100; // Cannot change

// finally block
try {
    int result = 10/0;
} catch(Exception e) {
    System.out.println("Exception");
} finally {
    System.out.println("Always executes"); // Cleanup code
}

// finalize method (deprecated)
protected void finalize() {
```

```
// Cleanup before GC (not recommended now)  
}
```

 **Note:** `finalize()` is deprecated from Java 9 onwards. Use `try-with-resources` or `Cleaner` class instead.

Volatile vs Synchronized

Question 5

Explain the difference between volatile keyword and synchronized blocks.

Feature	volatile	synchronized
Purpose	Visibility guarantee	Visibility + Atomicity
Lock	No locking	Acquires lock
Thread blocking	No	Yes (threads wait)
Performance	Faster	Slower
When to use	Single read/write	Multiple operations

Code Example:

```
// volatile - ensures all threads see latest value
private volatile boolean flag = true;

// synchronized - ensures atomic execution
public synchronized void increment() {
    count++; // Thread-safe
}

// synchronized block (finer control)
public void update() {
    synchronized(this) {
        // critical section
    }
}
```

```
}  
}
```

📌 **Key Point:** volatile only guarantees visibility, not atomicity. For compound operations (like `count++`), use `synchronized` or `AtomicInteger`.

Method Overloading vs Overriding

Question 6

Explain Method Overloading vs Overriding with examples. Can we change access modifier when overriding?

Feature	Method Overloading	Method Overriding
Definition	Same class, same name, different parameters	Child class redefines parent method
Return type	Can be different	Must be same or covariant
Parameters	Must be different	Must be same
Access modifier	Can be any	Cannot reduce visibility
Static	Can overload static	Cannot override static

Code Example:

```
// Overloading
class Calculator {
    int add(int a, int b) { return a+b; }
    int add(int a, int b, int c) { return a+b+c; } // Different
parameters
    double add(double a, double b) { return a+b; } // Different type
}
```

```
// Overriding  
class Parent {  
    protected void show() { System.out.println("Parent"); }  
}  
class Child extends Parent {  
    @Override  
    public void show() { System.out.println("Child"); } // Same  
signature, wider access  
}
```

Can we change access modifier when overriding?

- Yes, but cannot reduce visibility
- protected → public (OK)
- public → private (Not allowed)

Access Modifiers in Inheritance

Question 7

If Class A has private, protected, default, public methods - which can be overridden in different package?

Modifier	Same Class	Same Package	Different Package (Subclass)	Different Package (Non-subclass)
private	✓	✗	✗	✗
default	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

Answer: Only **protected and public** methods can be overridden in a different package.

Code Example:

```
// Package pack1
package pack1;
public class A {
    private void m1() { }
    void m2() { } // default
    protected void m3() { }
    public void m4() { }
```


Heap vs Stack Memory

Question 8

What is the difference between Heap and Stack?
What specific info is stored in each?

Aspect	Stack	Heap
What's stored	Local variables, method calls, references	Objects, instance variables, arrays
Memory type	Fixed size per thread	Shared across threads
Access	LIFO (Last In First Out)	Random access
Lifetime	Until method completes	Until garbage collected
Speed	Faster	Slower
Thread-safety	Thread-safe (each thread has own stack)	Not thread-safe

Code Example:

```
public void method() {  
    int x = 10; // Stored in Stack  
    Person p = new Person(); // p reference in Stack, Person object  
    in Heap  
}  
  
// Memory visualization
```

```
// Stack: [x:10] [p:reference]  
// Heap: [Person object at address 0x1234]
```

💡 **Interview Tip:** Draw the memory diagram to impress interviewer. Show stack frames and heap objects.

PermGen vs Metaspace

Question 9

Explain the difference between PermGen and Metaspace. Why did Java 8 move to Metaspace?

Feature	PermGen (Java 7 and earlier)	Metaspace (Java 8+)
Location	Part of Heap	Native memory (outside Heap)
Size	Fixed size (-XX:MaxPermSize)	Automatically grows
OutOfMemoryError	Common due to fixed size	Rare (uses native memory)
GC performance	Full GC required	Better performance
Tuning	Difficult	Easier

Why the change?

- PermGen had fixed size - caused OOM errors
- Metaspace uses native memory - grows automatically
- Better garbage collection performance
- Removes artificial limitations

```
// JVM options
// PermGen (Java 7)
-XX:PermSize=64m -XX:MaxPermSize=256m
```


Object Creation Ways

Question 10

How many ways can you create an object? If a String is created with new, how many objects are created?

5 Ways to Create Objects:

```
// 1. Using new keyword
Person p1 = new Person();

// 2. Using Class.forName() - Reflection
Person p2 = (Person) Class.forName("Person").newInstance();

// 3. Using clone()
Person p3 = (Person) p1.clone();

// 4. Using deserialization
ObjectInputStream in = new ObjectInputStream(new
FileInputStream("data.obj"));
Person p4 = (Person) in.readObject();

// 5. Using newInstance() of Constructor class
Constructor<Person> constructor = Person.class.getConstructor();
Person p5 = constructor.newInstance();
```

String with new:

```
String s = new String("Hello");
// Creates TWO objects:
// 1. In Heap (due to new)
// 2. In String Constant Pool (if "Hello" not already present)
```

 **Note:** String literals like "Hello" are stored in String Constant Pool for reuse.

String Immutability

Question 11

Why are Strings immutable? Explain the String Constant Pool.

Why String is Immutable:

- **Security** - Don't want passwords to change
- **Caching** - Can safely cache hashcodes
- **Thread-safe** - No synchronization needed
- **String Pool** - Enables string pooling

String Constant Pool:

- Special memory area in Heap
- Stores unique string literals
- Saves memory by reusing strings

```
String s1 = "Hello"; // Goes to String Pool
String s2 = "Hello"; // Reuses from pool
String s3 = new String("Hello"); // Creates new object in Heap

System.out.println(s1 == s2); // true (same reference)
System.out.println(s1 == s3); // false (different references)
System.out.println(s1.equals(s3)); // true (same content)
```

String Pool (Heap)

"Hello"
"World"
"Java"

← s1, s2 point here

Heap (outside pool)

String object
with "Hello"

← s3 points here

Abstract Class vs Interface

Question 12

Why can we not create an object of an abstract class? Can we have non-abstract methods in an abstract class? Can we create parameterized constructor in abstract class?

Feature	Abstract Class	Interface
Keyword	abstract	interface
Multiple inheritance	No	Yes
Instance variables	Can have	Only static final
Constructors	Yes	No
Methods	Abstract + concrete	abstract, default, static

Answers:

- **Why can't we create object of abstract class?** Abstract class is incomplete (may have abstract methods). Objects can only be created of concrete classes.
- **Can we have non-abstract methods in abstract class?** Yes, abstract class can have both abstract and concrete methods.
- **Can we create parameterized constructor in abstract class?** Yes. Called when concrete class object is created using `super()`.

Code Example:

Garbage Collection

Question 13

Explain GC implementation and the algorithms used (e.g., G1GC).

What is GC?

Automatic memory management that removes unused objects.

GC Algorithms:

Algorithm	How it works	Pros/Cons
Mark & Sweep	Marks live objects, sweeps dead ones	Simple but causes fragmentation
Copying	Copies live objects to new space	No fragmentation, but double memory
Mark & Compact	Marks, then compacts live objects	No fragmentation, slower
Generational	Young + Old generation separation	Most efficient

G1GC (Default since Java 9):

- Divides heap into regions
- Prioritizes regions with most garbage
- Predictable pause times

```
// Can suggest GC (not guaranteed)
System.gc();
```

```
// JVM options for GC
-XX:+UseG1GC // Use G1GC
-XX:MaxGCPauseMillis=200 // Target max pause time
```

 **Note:** `System.gc()` is just a request - JVM may ignore it.

Wrapper Classes

Question 14

Why do we need wrapper classes? Compare Wrapper vs Primitive.

Why Wrapper Classes?

- **Collections** - Can't store primitives in Collections
- **Generics** - Require objects, not primitives
- **Null values** - Primitives can't be null
- **Utility methods** - `parseInt()`, `valueOf()`, etc.

Feature	Primitive	Wrapper
Examples	int, double, boolean	Integer, Double, Boolean
Default value	0, 0.0, false	null
Memory	Less	More
Performance	Faster	Slower
Stored in	Stack	Heap

Code Example:

```
int x = 10; // Primitive
Integer y = 10; // Autoboxing - converts int to Integer automatically
int z = y; // Unboxing - converts Integer to int automatically

// Utility methods
```


Generics

Question 15

What are Generics in Java, and why are they used?

What are Generics?

Enable types (classes/interfaces) to be parameters when defining classes, interfaces, and methods.

Benefits:

- **Type safety** - Compile-time checking
- **No casting** - Avoid explicit casting
- **Code reusability** - Write once, use with multiple types

Code Example:

```
// Without Generics - need casting
List list = new ArrayList();
list.add("Hello");
String s = (String) list.get(0); // Casting needed

// With Generics - no casting
List<String> list = new ArrayList<>();
list.add("Hello");
String s = list.get(0); // No casting, type safe

// Generic class
class Box<T> {
    T value;
    void set(T value) { this.value = value; }
    T get() { return value; }
}
```

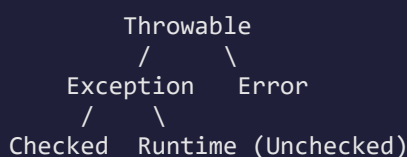

Exception Handling

Question 16

What is the superclass of all Exceptions? Explain try-with-resources vs traditional try-catch. What are the methods `printStackTrace()` and `getMessage()`?

Superclass of all Exceptions:

- **Throwable** is the superclass
- Two main branches: **Exception** and **Error**



Try-with-resources vs Traditional:

Feature	Traditional try-catch	Try-with-resources
Resource closing	Manual in finally	Automatic
Code length	Longer	Shorter
Multiple resources	Nested finally blocks	Single try block
Introduced	Java 1.0	Java 7

Code Example:

```
// Traditional
FileInputStream fis = null;
try {
    fis = new FileInputStream("file.txt");
    // read file
} catch(IOException e) {
    e.printStackTrace();
} finally {
    if(fis != null) {
        try { fis.close(); } catch(IOException e) { }
    }
}

// Try-with-resources (auto-close)
try(FileInputStream fis = new FileInputStream("file.txt")) {
    // read file - fis automatically closed
} catch(IOException e) {
    e.printStackTrace();
}
```

printStackTrace() vs getMessage():

- **printStackTrace():** Prints full stack trace (class, method, line numbers)
- **getMessage():** Returns only the error message string

PART 2

Java 8+ Features

Streams, Lambda, Collections & More

This section covers 15+ essential Java 8+ interview questions

↓ 15+ Questions Inside ↓

Java 8 Advantages

Question 1

Why did companies migrate from Java 7 to 8? List the key features.

Why Java 8 was game-changing:

- **Functional Programming** - Lambda expressions
- **Stream API** - Declarative data processing
- **Optional class** - Null safety
- **Date/Time API** - Better than `java.util.Date`
- **Default methods** - Interfaces with implementations
- **Performance** - Metaspace, better GC

Code Example:

```
// Before Java 8 - Anonymous inner class (verbose)
new Thread(new Runnable() {
    public void run() {
        System.out.println("Hello");
    }
}).start();

// After Java 8 - Lambda (concise)
new Thread(() -> System.out.println("Hello")).start();
```

Functional Interfaces

Question 2

How do you create a Functional Interface? What are the advantages? List common ones.

What is Functional Interface?

Interface with exactly one abstract method.

How to create:

```
@FunctionalInterface // Optional but recommended
interface Calculator {
    int calculate(int a, int b); // Single abstract method

    // Can have default methods
    default void show() {
        System.out.println("Calculator");
    }

    // Can have static methods
    static void info() {
        System.out.println("Functional Interface");
    }
}
```

Common Built-in Functional Interfaces:

Interface	Method	Purpose	Example
Predicate<T>	boolean test(T t)	Test condition	s -> s.length() > 5

Function<T,R>	R apply(T t)	Transform input	s -> s.length()
Consumer<T>	void accept(T t)	Consume input	s -> System.out.println(s)
Supplier<T>	T get()	Supply value	() -> new Date()

Code Example:

```
// Predicate
Predicate<String> isLong = s -> s.length() > 5;
System.out.println(isLong.test("Hello")); // false

// Function
Function<String, Integer> getLength = s -> s.length();
System.out.println(getLength.apply("Java")); // 4

// Consumer
Consumer<String> print = s -> System.out.println(s);
print.accept("Hello"); // Hello

// Supplier
Supplier<Double> random = () -> Math.random();
System.out.println(random.get());
```

Functional vs Marker Interface

Question 3

Explain the difference between Functional Interface and Marker Interface with examples.

Feature	Functional Interface	Marker Interface
Methods	One abstract method	No methods
Purpose	Lambda expressions	Mark capability
Example	Runnable, Comparator	Serializable, Cloneable
Usage	@FunctionalInterface	instanceof check

Code Example:

```
// Functional Interface
@FunctionalInterface
interface MyFunctional {
    void execute(); // One method
}

// Marker Interface
interface MyMarker { // No methods
}

class Test implements MyMarker {
    // No methods to implement
}
```


Lambda Expressions

Question 4

How do Lambda Expressions simplify code? Provide examples.

Lambda Expression Syntax:

```
(parameters) -> expression  
(parameters) -> { statements; }
```

Before Lambda (Anonymous class):

```
// Sorting with Comparator  
Collections.sort(list, new Comparator<String>() {  
    @Override  
    public int compare(String s1, String s2) {  
        return s1.length() - s2.length();  
    }  
});  
  
// Runnable  
new Thread(new Runnable() {  
    public void run() {  
        System.out.println("Thread running");  
    }  
}).start();
```

After Lambda:

```
// Sorting - one line
Collections.sort(list, (s1, s2) -> s1.length() - s2.length());

// Runnable - concise
new Thread(() -> System.out.println("Thread running")).start();

// Event listener
button.addActionListener(e -> System.out.println("Clicked"));

// Multiple parameters
Comparator<String> comp = (s1, s2) -> {
    System.out.println("Comparing");
    return s1.length() - s2.length();
};
```

💡 **Lambda Benefits:** Concise code, improved readability, enables functional programming, easier parallel processing.

Optional Class

Question 5

How do you use Optional? Which inbuilt methods have you used to avoid null checks?

What is Optional?

Container that may or may not contain a value - avoids NullPointerException.

Common Methods:

Method	Purpose
of(value)	Create Optional with non-null value
ofNullable(value)	Create Optional that can be null
empty()	Create empty Optional
isPresent()	Check if value exists
ifPresent(consumer)	Execute if value exists
orElse(default)	Return value or default
orElseGet(supplier)	Return value or compute
orElseThrow(exception)	Return value or throw

Code Example:

```
// Without Optional - risk of NPE
public String getCity(User user) {
```


Stream API - map vs flatMap

Question 6

Explain map() vs flatMap(). What are intermediate vs terminal operations?

map() vs flatMap():

Operation	map()	flatMap()
Purpose	Transform each element	Transform + flatten
Input	Stream<T>	Stream<T>
Output	Stream<R>	Stream<R>
Relationship	One-to-one	One-to-many
Use case	Simple transformation	Flatten nested structures

Code Example:

```
// map - each element becomes one element
List<String> words = Arrays.asList("hello", "world");
List<Integer> lengths = words.stream()
    .map(String::length)
    .collect(Collectors.toList()); // [5, 5]

// flatMap - each element can become multiple elements
List<String> letters = words.stream()
    .flatMap(s -> Arrays.stream(s.split("")))
    .collect(Collectors.toList()); // [h,e,l,l,o,w,o,r,l,d]
```

Intermediate vs Terminal Operations:

Type	Description	Examples	Execution
Intermediate	Return Stream, lazy	filter, map, sorted, distinct	Not executed until terminal
Terminal	Return result, eager	collect, forEach, count, reduce	Triggers execution

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);

// Pipeline: Source → Intermediate → Intermediate → Terminal
List<Integer> result = numbers.stream() // Source
    .filter(n -> n % 2 == 0) // Intermediate (lazy)
    .map(n -> n * n) // Intermediate (lazy)
    .collect(Collectors.toList()); // Terminal (executes)

// filter and map don't run until collect is called!
```

Stream vs Parallel Stream

Question 7

Explain Stream vs Parallel Stream.

Feature	Stream	Parallel Stream
Processing	Single thread	Multiple threads
Order	Maintains encounter order	May not maintain order
Performance	Slower for large data	Faster for large data
Overhead	Minimal	Thread management overhead
When to use	Small data, ordered required	Large data, independent operations

Code Example:

```
List<Integer> numbers = IntStream.rangeClosed(1, 1000000)
    .boxed().collect(Collectors.toList());

// Sequential stream
long start = System.currentTimeMillis();
numbers.stream()
    .filter(n -> n % 2 == 0)
    .count();
System.out.println("Sequential: " + (System.currentTimeMillis() -
start));

// Parallel stream
start = System.currentTimeMillis();
numbers.parallelStream()
    .filter(n -> n % 2 == 0)
```

```
        .count();  
  
        System.out.println("Parallel: " + (System.currentTimeMillis() -  
        start));  
  
        // For large data, parallel is faster!
```

When NOT to use parallel stream:

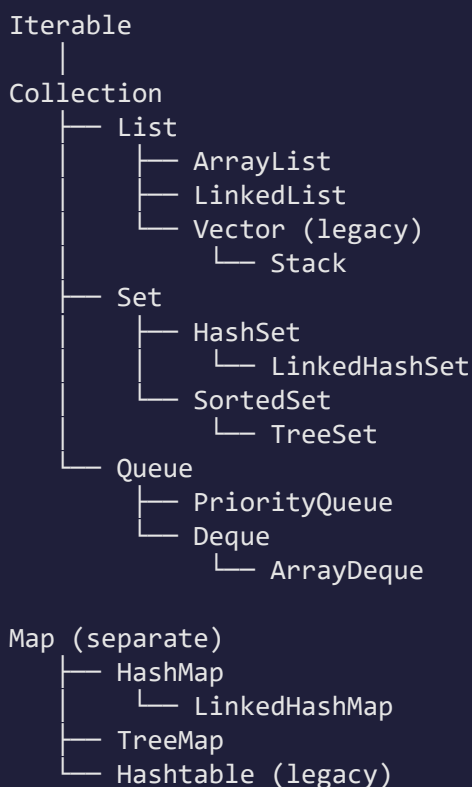
- Small datasets (overhead > benefit)
- Ordered operations required
- Non-thread-safe operations
- I/O operations (file, network)

Collections Framework Hierarchy

Question 8

Explain the Collections Framework hierarchy.
Difference between ArrayList and LinkedList.

Collection Hierarchy:



ArrayList vs LinkedList:

Feature	ArrayList	LinkedList
Data structure	Dynamic array	Doubly-linked list

get(index)	$O(1)$ - Fast	$O(n)$ - Slow
add(end)	$O(1)^*$	$O(1)$
add(index)	$O(n)$ - Shifting	$O(1)$ - Link change
remove(index)	$O(n)$ - Shifting	$O(1)$ - Link change
Memory	Less	More (prev/next pointers)

Why is insertion faster in LinkedList?

- ArrayList needs to shift elements when inserting in middle
- LinkedList just changes the links between nodes - no shifting!

```
// LinkedList - fast insertion in middle
LinkedList<String> list = new LinkedList<>();
list.add("A"); list.add("C");
list.add(1, "B"); // Just changes links between A and C
```

HashMap Internal Working

Question 9

Explain HashMap internal working, put() method, and collision handling.

HashMap Internal Structure:

- Array of buckets (default capacity 16)
- Each bucket is a linked list (or tree after threshold)
- Load factor: 0.75 (default)

put() method workflow:

1. Calculate hash of key using hashCode()
2. Find bucket index: $\text{index} = \text{hash} \& (\text{n}-1)$
3. If bucket empty → create new node
4. If collision occurs → add to linked list/tree
5. Check for existing key using equals()
6. If key exists → replace value
7. If threshold exceeded → resize (double capacity)

Collision Handling:

- **Chaining** - Multiple entries in same bucket (linked list)
- After 8 collisions, list converts to **Red-Black tree** (Java 8+)
- After 6, tree converts back to list

```
Bucket 0: null
Bucket 1: [Key1:Value1] → [Key5:Value5] // Collision - linked list
Bucket 2: [Key2:Value2]
Bucket 3: null
```

...

```
HashMap<String, Integer> map = new HashMap<>();  
map.put("Apple", 10); // hash → bucket index → store  
map.put("Banana", 20);  
map.put("Apple", 30); // Key exists → value replaced
```

HashMap vs ConcurrentHashMap vs WeakHashMap

Question 10

Difference between HashMap, ConcurrentHashMap, and WeakHashMap.

Feature	HashMap	ConcurrentHashMap	WeakHashMap
Thread-safety	No	Yes	No
Null keys	Yes (one)	No	Yes
Null values	Yes	No	Yes
Performance	Fast	Medium	Fast
Key references	Strong	Strong	Weak
Use case	General purpose	Multi-threaded	Caches

Code Example:

```
// HashMap - not thread-safe
Map<String, String> map = new HashMap<>();

// ConcurrentHashMap - thread-safe, no nulls
Map<String, String> concurrentMap = new ConcurrentHashMap<>();
concurrentMap.put("key", "value"); // OK
// concurrentMap.put(null, "value"); // NullPointerException!
```


Fail-Fast vs Fail-Safe Iterators

Question 11

Explain the difference between Fail-Fast and Fail-Safe iterators.

Feature	Fail-Fast	Fail-Safe
Exception	Throws ConcurrentModificationException	No exception
Works on	Original collection	Copy of collection
Performance	Faster	Slower (copy overhead)
Memory	Less	More
Examples	ArrayList, HashMap	ConcurrentHashMap, CopyOnWriteArrayList

Code Example:

```
// Fail-Fast - throws exception
List<String> list = new ArrayList<>();
list.add("A"); list.add("B"); list.add("C");

Iterator<String> it = list.iterator();
while(it.hasNext()) {
    System.out.println(it.next());
    list.add("D"); // Throws ConcurrentModificationException!
}

// Fail-Safe - no exception
ConcurrentHashMap<String, String> map = new ConcurrentHashMap<>();
```


HashMap vs TreeSet

Question 12

Compare HashMap and TreeSet performance and ordering.

Feature	HashMap	TreeSet
Data structure	Hash table	Red-Black tree
Elements	Key-Value pairs	Single elements
Order	No order	Sorted
Null allowed	Yes (one key)	No (after first element)
Performance	O(1) average	O(log n)
When to use	Fast lookup	Need sorted unique elements

```
// HashMap - fast, unordered
HashMap<Integer, String> map = new HashMap<>();
map.put(5, "five");
map.put(2, "two");
map.put(8, "eight");
System.out.println(map); // Random order

// TreeSet - sorted, unique
TreeSet<Integer> set = new TreeSet<>();
```


Comparable vs Comparator

Question 13

Difference between Comparable and Comparator.

Feature	Comparable	Comparator
Package	java.lang	java.util
Method	compareTo(T o)	compare(T o1, T o2)
Modifies class	Yes (class implements)	No (separate class)
Sorting sequences	Single (natural order)	Multiple (custom orders)
When to use	Default sorting	Custom sorting

Code Example:

```
// Comparable - natural ordering
class Student implements Comparable<Student> {
    int id; String name;
    public int compareTo(Student s) {
        return this.id - s.id; // Sort by id
    }
}

// Comparator - custom ordering
Comparator<Student> byName = (s1, s2) ->
    s1.name.compareTo(s2.name); // Sort by name

Comparator<Student> byIdDesc = (s1, s2) ->
    s2.id - s1.id; // Sort by id descending
```


Thank You!

Complete Core Java & Java 8+

Interview Questions Guide

Shaikh Shakir

Java Developer

Happy Coding! 🚀